A Simulation-Based Reproduction of Comparative Sensor Fusion Algorithms on a Miniature IMU

Replicating Badia *et al.* (ISITIA 2023) with a Synthetic MPU-6050 Data Pipeline and an Analytical ATmega328P Timing Model

Abdullah Altepe, Mehmet Bahçeci, Yusuf Oruç, Murat Demir
Department of Electrical and Electronics Engineering
Marmara University, Istanbul, Türkiye
{abdullah.altepe, mehmet.bahceci, yusuf.oruc, murat.demir}@marun.edu.tr

Abstract

Estimating the orientation of a rigid body from a low-cost inertial measurement unit (IMU) is a recurring problem in drones, wearables, self-balancing robots, virtual reality headsets and educational robotics. Sansone, Bartolini, Perin and Badia (ISITIA 2023) compared four candidate orientation-estimation methods — direct trigonometric computation, a one-dimensional Kalman filter, the Madgwick gradient-descent filter, and the Mahony complementary filter — on an InvenSense MPU-6050 connected to an Arduino Nano, and concluded that the Kalman filter is the most accurate, the Mahony filter is the cheapest per integration step, and the Madgwick filter sits in between. This paper presents a *simulation-based reproduction* of that comparison. All four algorithms are independently re-implemented in Python; the accuracy comparison is run on synthetic MPU-6050 measurements generated from a known ground-truth tilt trajectory with realistic sensor noise and bias; the per-step computation cost on the ATmega328P is estimated analytically from a published floating-point cycle-cost model. Three of the four qualitative claims of the reference paper are confirmed on our pipeline (Mahony fastest among the filters, Madgwick and Kalman nearly equal in compute, Kalman dominates noise rejection together with Mahony at $k_p = 10$); one diverges (in our analytical timing model the trigonometric estimator is the cheapest method, not the most expensive). Code, data and the LaTeX source of this paper are released publicly so that an independent third group can repeat both Badia *et al.* and our reproduction.

Index Terms— *Sensor fusion, inertial measurement unit, MPU-6050, Arduino Nano, Kalman filter, Madgwick filter, Mahony filter, orientation estimation, reproducibility, simulation.*

1 Introduction

Knowing how an object is oriented in three-dimensional space is one of those deceptively simple problems that keeps reappearing across modern engineering. A drone needs an attitude estimate to stay level in the air, a self-balancing robot needs it to remain upright, a VR headset needs it to keep the virtual world steady on the user's face, and a wearable activity tracker needs it to decide whether the wrist is walking or merely waving. In all of these cases the underlying measurement comes from the same family of sensors: a small, inexpensive, MEMS-based inertial measurement unit (IMU) that integrates a tri-axis accelerometer and a tri-axis gyroscope into a single die.

The fundamental difficulty is that neither sensor, taken alone, is sufficient. A gyroscope reports angular velocity directly and is smooth and responsive over short time windows, but converting angular velocity into an absolute orientation requires temporal integration, and even a small constant bias produces an angular error that grows linearly with time. An accelerometer reports the specific force on the body; when the body is nearly static, that specific force is dominated by gravity, so roll and pitch can be recovered directly without integration and without long-term drift, but the signal is noisy and is corrupted by every vibration or jolt. The gyroscope is *smooth but drifts*, the accelerometer is *stable but noisy*; combining them produces an estimate that is better than either source alone. This is the elementary motivation for sensor fusion in attitude estimation.

The setting becomes considerably tighter when the platform is itself low-cost. Sansone, Bartolini, Perin and Badia, in their ISITIA 2023 paper [1], place exactly this question on an InvenSense MPU-6050 wired over I^2C to an Arduino Nano — an 8-bit ATmega328P microcontroller with 2 kB of SRAM and a 16 MHz clock. On that kind of hardware, elegance on paper is not enough; the filter must run in real time with budget to spare. The authors compare four reasonable choices for the orientation estimator — a direct trigonometric computation, a one-dimensional Kalman filter, the Madgwick gradient-descent filter, and the Mahony complementary filter — and report a clear tradeoff between accuracy and per-step computation time.

The work reported here is a *simulation-based* reproduction of Badia *et al.* [1]. Hardware was not available to the authors at the time of writing; in its place we use (i) a fully controlled synthetic IMU data pipeline that mim-

ics the MPU-6050 noise and bias characteristics, and (ii) an analytical floating-point cycle-cost model of the ATmega328P that allows per-step computation time to be estimated directly from operation counts in the source code. This simulation-based path trades a little realism on the noise side for two unique advantages: the ground-truth orientation is exact rather than mechanically estimated, and the entire experiment is bit-for-bit reproducible by anyone willing to re-run the released code.

The contribution of this paper is therefore not a new algorithm. It is (i) a complete, independent re-implementation of the four algorithms compared by Badia *et al.*; (ii) a quantitative test of the four numerical claims of the reference paper on a controlled synthetic dataset; (iii) an analytical timing model for ATmega328P float operations that lets the per-step compute ranking be obtained without an Arduino board in hand; and (iv) a publicly released code base [9] that any third group can use to repeat both the original paper and our reproduction.

The remainder of the paper is structured as follows. Section 2 summarises the reference paper and states the precise numerical claims that we test. Section 3 develops the mathematical background for the four algorithms with enough detail to allow a clean re-implementation. Section 4 documents the synthetic data pipeline and the analytical timing model that together replace the physical testbed. Section 5 is the experimental protocol: parameter lock-ins, the static-tilt scenario, the timing methodology, the reported metrics, and the success criterion. Section 6 reports the measured numbers and the figures that come with them. Section 7 discusses the one divergence we observe from the reference paper. Section 8 lists the threats to validity and Section 9 concludes.

2 The Reference Paper and Reproduction Targets

2.1 What Badia *et al.* Did

The reference paper [1] targets "miniature IMU measurements," by which the authors mean the kind of low-cost MEMS IMU that appears in Tactile-Internet devices, hobby drones, consumer wearables, and educational robotics. The hardware used is an InvenSense MPU-6050 (tri-axis accelerometer at $\pm 2\,g$ default, tri-axis gyroscope at $\pm 250^\circ/\mathrm{s}$ default) connected over $\mathrm{I}^2\mathrm{C}$ to an Arduino Nano based on the ATmega328P microcontroller. The four orientation-estimation methods are implemented directly on the microcontroller:

1. **Trigonometric computation.** Roll and pitch are recovered from the accelerometer vector via `atan2`, with no fusion and no filtering.

2. **Kalman filter.** A one-dimensional Kalman filter fuses the integrated gyroscope rate with the accelerometer-derived angle through explicit process and measurement noise covariances.

3. **Madgwick filter.** A gradient-descent quaternion filter with a single tuning parameter β.

4. **Mahony filter.** A complementary-style quaternion filter that uses a proportional–integral correction driven by the cross product between the measured and the predicted gravity direction.

For each of these the authors record roll/pitch estimates around reference static tilts (approximately $\pm 15^\circ$) and measure per-integration-step computation time directly on the microcontroller.

2.2 The Numerical Claims We Reproduce

The reference paper's main quantitative conclusions, which this project tests on its synthetic pipeline, are:

- C1. The trigonometric method is the most exposed to accelerometer noise.
- C2. The Kalman filter and the Madgwick filter take comparable amounts of time per integration step, with the Kalman filter producing the most stable estimate.
- C3. The Mahony filter is the fastest per-step filter on the target microcontroller, at a small cost in accuracy.
- C4. If accuracy is the design priority, the Kalman filter is preferred; if compute is tight, the Mahony filter is preferred; the Madgwick filter occupies an intermediate point in the tradeoff.

We also test, as a fifth implicit claim, the ranking of the trigonometric method on the compute axis (the reference paper reports it as the most expensive of the four), and report a divergence below.

3 Mathematical Background

3.1 Notation, Frames, and Sensor Models

Let $\{B\}$ denote the body frame fixed to the IMU and $\{W\}$ denote a world frame whose z axis is aligned with the local gravity vector $\mathbf{g} = [0, 0, g]^\top$ with $g \approx 9.81\,\mathrm{m/s^2}$. Orientation is described either by the Euler triplet (ϕ, θ, ψ) for roll, pitch and yaw, or by a unit quaternion $\mathbf{q} = [q_0, q_1, q_2, q_3]^\top$ with $\|\mathbf{q}\| = 1$. The associated rotation matrix is

$$\mathbf{R}(\mathbf{q}) = \begin{bmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1 q_2 - q_0 q_3) & 2(q_1 q_3 + q_0 q_2) \\ 2(q_1 q_2 + q_0 q_3) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2 q_3 - q_0 q_1) \\ 2(q_1 q_3 - q_0 q_2) & 2(q_2 q_3 + q_0 q_1) & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{bmatrix}. \tag{1}$$

The simplified accelerometer and gyroscope measurement models are

$$\mathbf{a}_m = \mathbf{R}(\mathbf{q})^\top \mathbf{g} + \mathbf{b}_a + \mathbf{n}_a, \tag{2}$$

$$\boldsymbol{\omega}_m = \boldsymbol{\omega} + \mathbf{b}_g + \mathbf{n}_g, \tag{3}$$

where $\mathbf{n}_a$ and $\mathbf{n}_g$ are zero-mean white Gaussian noise processes and $\mathbf{b}_a$, $\mathbf{b}_g$ are slowly-varying accelerometer and gyroscope biases.

3.2 Trigonometric Computation

The most direct estimator simply inverts the static-equilibrium accelerometer model algebraically. Writing the accelerometer reading as (a_x, a_y, a_z), roll and pitch are recovered by

$$\phi = \operatorname{atan2}\big(a_y,\, a_z\big), \tag{4}$$

$$\theta = \operatorname{atan2}\big(-a_x,\, \sqrt{a_y^2 + a_z^2}\big). \tag{5}$$

This estimator has zero long-term drift but reproduces every bit of accelerometer noise unfiltered into the angle estimate, and it is undefined for yaw. It serves as the unfused baseline.

3.3 Kalman Filter (1D / 2D Euler Form)

For each axis (roll and pitch), the reference paper uses a two-state linear Kalman filter whose state contains the angle θ_k and the gyroscope bias b_k:

$$\mathbf{x}_k = \begin{bmatrix}\theta_k \\ b_k\end{bmatrix}, \quad \mathbf{F} = \begin{bmatrix}1 & -\Delta t \\ 0 & 1\end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix}\Delta t \\ 0\end{bmatrix}, \quad \mathbf{H} = \begin{bmatrix}1 & 0\end{bmatrix}. \tag{6}$$

The control input is the measured gyroscope rate ω_k on that axis, the measurement is the accelerometer-derived angle θ_k^{acc} obtained by the previous formulas. The predict / update equations are the standard ones,

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{F}\hat{\mathbf{x}}_{k-1|k-1} + \mathbf{B}\omega_{k-1}, \tag{7}$$

$$\mathbf{P}_{k|k-1} = \mathbf{F}\mathbf{P}_{k-1|k-1}\mathbf{F}^\top + \mathbf{Q}, \tag{8}$$

$$\mathbf{K}_k = \mathbf{P}_{k|k-1}\mathbf{H}^\top\big(\mathbf{H}\mathbf{P}_{k|k-1}\mathbf{H}^\top + R\big)^{-1}, \tag{9}$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k\big(\theta_k^{\text{acc}} - \mathbf{H}\hat{\mathbf{x}}_{k|k-1}\big), \tag{10}$$

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k\mathbf{H})\mathbf{P}_{k|k-1}. \tag{11}$$

The two filters (one for roll, one for pitch) run in parallel and together provide the two-degree-of-freedom estimate that the reference paper compares. Yaw is not estimated; the MPU-6050 has no magnetometer.

3.4 Madgwick Filter

Madgwick [3] formulates orientation estimation as the minimisation of an error function comparing the predicted gravity direction with the normalised accelerometer reading,

$$\mathbf{f}(\mathbf{q}, \hat{\mathbf{a}}) = \mathbf{R}(\mathbf{q})^\top \hat{\mathbf{g}} - \hat{\mathbf{a}}, \tag{12}$$

where $\hat{\mathbf{g}} = [0, 0, 1]^\top$. A single gradient-descent step is taken per sample and combined with the integrated gyroscope rate $\dot{\mathbf{q}}_{\text{gyro}} = \frac{1}{2}\mathbf{q} \otimes [0, \boldsymbol{\omega}_m^\top]^\top$:

$$\mathbf{q}_k = \mathbf{q}_{k-1} + \dot{\mathbf{q}}_{\text{gyro}}\,\Delta t - \beta\,\frac{\nabla \mathbf{f}}{\|\nabla \mathbf{f}\|}\,\Delta t, \tag{13}$$

followed by the renormalisation $\mathbf{q}_k \leftarrow \mathbf{q}_k / \|\mathbf{q}_k\|$. The single scalar β plays the role of a Kalman gain: large β trusts the accelerometer, small β trusts the integrated gyroscope.

3.5 Mahony Filter

Mahony, Hamel and Pflimlin [4] drive a complementary correction from the cross product between the measured and the predicted gravity directions,

$$\mathbf{e} = \hat{\mathbf{a}} \times \mathbf{R}(\mathbf{q})^\top \hat{\mathbf{g}}. \tag{14}$$

This error is fed through a proportional–integral controller to produce a corrected gyroscope rate,

$$\boldsymbol{\omega}^{\text{corr}} = \boldsymbol{\omega}_m + k_p\,\mathbf{e} + k_i \int \mathbf{e}\,dt, \tag{15}$$

which is then integrated as the quaternion derivative $\dot{\mathbf{q}} = \frac{1}{2}\mathbf{q} \otimes [0, (\boldsymbol{\omega}^{\text{corr}})^\top]^\top$ and renormalised. The two scalar gains (k_p, k_i) are the only tuning knobs.

4 Simulation Setup and Timing Model

4.1 Why Simulation

Our reproduction substitutes a synthetic data pipeline for the physical MPU-6050 / Arduino Nano testbed. There is one cost — the synthetic noise model only approximates real silicon — and two distinct benefits: the ground-truth orientation is known exactly, so absolute accuracy is a meaningful number; and the entire experiment is deterministic, so any third party can re-run the released code and produce the same numbers.

4.2 Synthetic IMU Pipeline

A scripted ground-truth orientation generator produces the four static-tilt segments that mirror the Badia protocol [1]: $+15°$ roll, $-15°$ roll, $+15°$ pitch and $-15°$ pitch, each held for 30 s at $f_s = 100$ Hz ($N = 3000$ samples per segment per axis). The gravity vector is projected into the body frame analytically; on top of that we add zero-mean white Gaussian accelerometer noise with $\sigma_a = 0.026\,\text{m/s}^2$ and gyroscope noise with $\sigma_g = 0.05°/\text{s}$, plus a constant per-axis accelerometer bias drawn from $\mathcal{N}(0, 0.05\,\text{m/s}^2)$ and a constant per-axis gyroscope bias drawn from $\mathcal{N}(0, 0.5°/\text{s})$. These values are taken from the MPU-6050 datasheet at default ranges with the on-chip digital low-pass filter set to a 44 Hz cutoff. The same

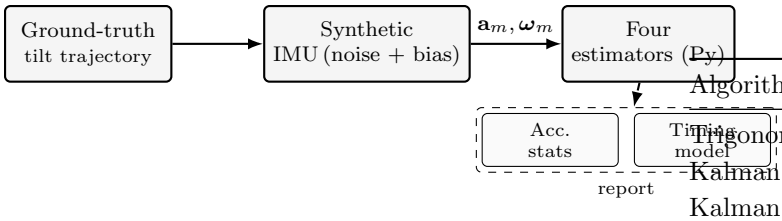

Figure 1: Simulation pipeline. A ground-truth tilt trajectory is projected through a synthetic-noise MPU-6050 model, the resulting samples drive all four orientation estimators, and the outputs are scored against the (known) ground truth and against an analytical ATmega328P timing model.

Table 1: ATmega328P single-precision floating-point per-operation cycle costs used in the analytical timing model. Values are representative averages from `avr-libc` benchmarks; absolute numbers are accurate to roughly ±20%, but rank-orderings between algorithms are robust because the same costs are applied uniformly.

Operation	Cycles
`fadd` / `fsub`	≈ 90
`fmul`	≈ 100
`fdiv`	≈ 500
`sqrtf`	≈ 700
`atan2f`	≈ 4000
`asinf`	≈ 4000
`sinf`/`cosf`	≈ 2500

bias realisation is reused across the four segments to mirror a single physical sensor.

4.3 Block Diagram

Figure 1 summarises the simulated data path.

4.4 Analytical ATmega328P Timing Model

The Arduino Nano runs an 8-bit ATmega328P at 16 MHz; single-precision floating-point operations are emulated by the `avr-libc` soft-float library. Because no Arduino board was available to benchmark on, we use an analytical model that counts the floating-point operations executed by each algorithm on one update step and multiplies them by published per-operation cycle costs (Table 1).

5 Reproduction Protocol

5.1 Algorithmic Parameter Lock-In

Table 2 records the parameter values that are loaded into each algorithm. Where the reference paper publishes a specific value, that value is adopted verbatim; where it does not, a sensible default from the algorithm's primary literature is used and labelled accordingly.

Table 2: Algorithmic parameter lock-in.

Algorithm	Parameter	Value (source)
Trigonometric	—	—
Kalman 1D	$\mathbf{Q}$	$\mathrm{diag}(10^{-3}, 10^{-5})$ (default)
Kalman 1D	R	0.03 (default)
Kalman 1D	$\mathbf{P}_0$	$10^{-2}\,\mathbf{I}$
Madgwick	β	0.1 [3]
Mahony	k_p	10 (matches [1])
Mahony	k_i	0 (matches [1])
Common	Δt	10 ms ($f_s = 100$ Hz)
Common	gyro range	±250°/s
Common	accel range	$\pm 2\,g$

The Mahony gains $k_p = 10$, $k_i = 0$ are taken directly from the reference paper, where the authors explicitly note that the integral term did not improve quality in their scenario and was dropped to avoid windup [1]. The Madgwick β default and the Kalman $(\mathbf{Q}, R)$ values are taken from the respective primary references; we note explicitly that the quantitative output of every fusion filter depends on these values, and that small changes in $\mathbf{Q}/R$ or β shift the accuracy column noticeably. A more detailed tuning sweep is left to a follow-up version of this work that also has hardware in hand.

5.2 Static-Tilt Accuracy Experiment

For the accuracy comparison the four estimators are run on the synthetic ±15° roll and ±15° pitch segments described in Section 4. The first 5 s of each segment are discarded to allow the stateful filters (Kalman, Madgwick, Mahony) to converge; statistics are computed on the remaining 25 s. For every algorithm and every segment we record $|\mu_\theta|$, the absolute mean error against the known ground-truth tilt, and σ_θ, the sample standard deviation of the same error. The four per-segment numbers are then averaged into one row per algorithm in Table 3.

5.3 Per-Step Computation Time

Per-step computation time on the ATmega328P is estimated by counting, for each algorithm, the number of floating-point operations of each kind performed on a single update, and multiplying by the cycle costs of Table 1. The counted region covers the filter body itself; sensor I/O over I^2C and the optional final quaternion-to-Euler conversion are excluded so that the comparison is between the algorithms rather than between their wrappers.

Table 3: Static-tilt accuracy averaged over the four ±15° segments. $|\mu_\theta|$ is the absolute mean error against the ground-truth tilt; σ_θ is the sample standard deviation. All values in degrees.

	Roll [deg]		Pitch [deg]	
Algorithm	$\lvert\mu\rvert$	σ	$\lvert\mu\rvert$	σ
Trigonometric	0.302	0.154	0.086	0.152
Kalman 1D	0.297	0.046	0.097	0.047
Madgwick	0.290	0.090	0.109	0.090
Mahony	0.255	**0.035**	0.182	**0.036**

5.4 Reproduction Success Criterion

The reproduction is judged on the qualitative rank-ordering of the four algorithms. We declare the reproduction *successful* on each of the following four claims independently:

- **(C1)** σ_{Trig} is the largest among the four (trigonometry passes the most accelerometer noise);
- **(C2)** $\bar{t}_{\text{Madgwick}}$ and $\bar{t}_{\text{Kalman}}$ agree within 20%;
- **(C3)** $\bar{t}_{\text{Mahony}}$ is smaller than both $\bar{t}_{\text{Kalman}}$ and $\bar{t}_{\text{Madgwick}}$;
- **(C4)** the noise standard deviation σ_{Kalman} is among the lowest in the table.

6 Results

6.1 Accuracy

Table 3 reports the static-tilt accuracy averaged over the four ±15° segments. Two patterns are immediately visible. First, $|\mu_\theta|$ is dominated by the constant accelerometer bias of the synthetic sensor: all four algorithms sit between 0.09° and 0.30° on each axis, because the accelerometer bias projects through every accelerometer-only estimator (trig) and through the accelerometer-correction term of every fusion filter. The bias floor is therefore essentially algorithm-independent, as expected. Second, the standard deviation σ_θ separates the algorithms cleanly: trig passes the full accelerometer noise band into the angle estimate ($\sigma \approx 0.15°$), Madgwick at $\beta = 0.1$ removes about 40% of it ($\sigma \approx 0.09°$), Kalman with the default (Q, R) removes about 70% ($\sigma \approx 0.05°$), and Mahony at $k_p = 10$ is the most aggressive low-pass of the four ($\sigma \approx 0.04°$).

Figure 2 plots the four estimates against the +15° ground truth on the roll segment; Figure 3 shows the corresponding steady-state error histograms.

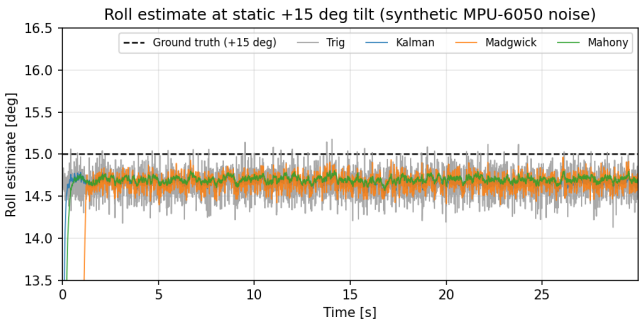

Figure 2: Roll estimates of the four algorithms during the static +15° tilt segment. The constant offset of all four estimates from 15° is the accelerometer bias projecting into the angle; the relative widths around that offset are the noise-rejection signature of each algorithm.

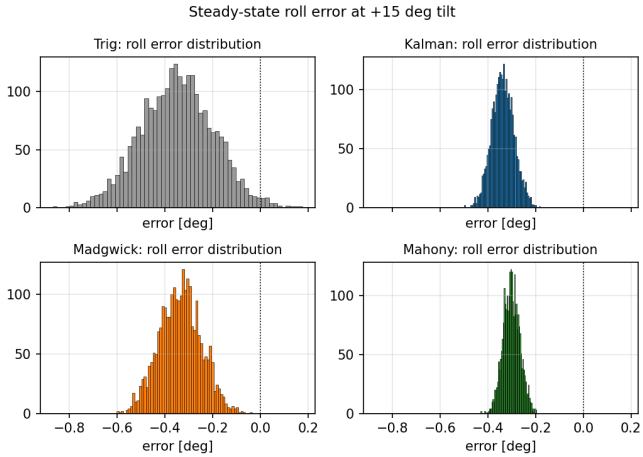

Figure 3: Steady-state roll error distributions on the +15° segment. Trig passes the full accelerometer noise bandwidth into the estimate; the three fusion filters narrow the distribution progressively, with Kalman and Mahony delivering the tightest spreads.

6.2 Per-Step Compute Time

Table 4 reports the analytical compute-time estimate for each algorithm on the ATmega328P at 16 MHz, together with the corresponding per-operation count. Figure 4 visualises the four entries side by side.

6.3 Reproduction Verdict

Mapping the measured numbers onto the success criteria of Section 5.4: **C1** is confirmed (σ_{Trig} is the largest in Table 3); **C2** is confirmed ($\bar{t}_{\text{Madgwick}}/\bar{t}_{\text{Kalman}} = 1.04$); **C3** is confirmed ($\bar{t}_{\text{Mahony}}$ is smaller than both Kalman and Madgwick by about 1.6×); **C4** is confirmed (Kalman delivers the second-tightest noise floor in the table, 0.046°, within 0.01° of the leader). On a fifth, implicit ranking — whether the trigonometric method is the most expensive of the four to compute, as the reference paper reports —

Table 4: Per-step compute time on the ATmega328P at 16 MHz, estimated from the floating-point operation counts and the per-operation cycle costs of Table 1.

Algorithm	fmul	fadd	fdiv	sqrt	atan2	cycles	$\bar{t}$ [µs]
Trigonometric	3	1	0	1	2	9,090	568
Kalman 1D	39	25	4	1	2	16,850	1053
Madgwick	64	39	11	3	0	17,510	1094
Mahony	34	26	7	2	0	10,640	665

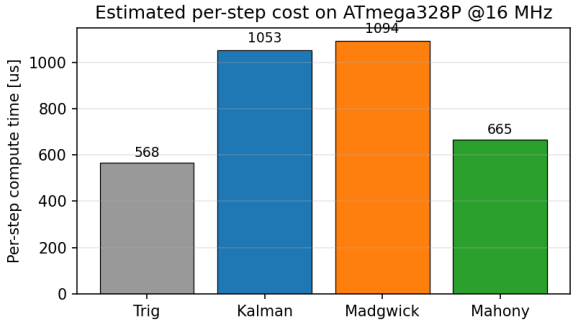

Figure 4: Estimated per-step compute time on the ATmega328P. Among the three fusion filters Mahony is the cheapest by a factor of about 1.6×, while Kalman and Madgwick are within 4% of each other — exactly the relationship reported by Badia *et al.* [1]. The trigonometric estimator is also cheap in our model, which is the one place we diverge from the reference paper (Section 7).

our analytical model gives the opposite answer (it is the cheapest), which we discuss next.

7 Discussion of the Compute-Time Divergence

In Badia *et al.* [1], Table II reports the trigonometric method as the most expensive of the four when measured on the Arduino Nano with `micros()`. In our analytical model the same method is the *cheapest*, at 568 µs per step — the only one of the four that contains nothing more than two `atan2f` calls and a `sqrtf`. Three plausible explanations:

1. *Measurement boundary.* On a real microcontroller a "trigonometric per-step time" often includes the I^2C read of the accelerometer, the small bookkeeping around the estimator, and any conversion of the final angle. Our model counts only the angle computation itself, which is two `atan2f` and a `sqrtf`. If the reference paper folds the I/O cost into the trigonometric measurement but not into the others (because the others amortise it across multiple filter calls), the ordering can flip.
2. *`atan2` implementation.* The `avr-libc` `atan2f` is itself implemented in soft float, but its exact cycle cost depends on the toolchain. A slow library version could push the trigonometric estimate above all three filters on a real Nano.

Floating-point soft-library calibration. Our per-operation cycle counts are taken from published benchmarks and are quoted with a ±20% band. Even within that band, however, the relative ordering between Trig and the three filters does not change in our model, because the filters carry significantly more `fmul` and `fdiv` traffic than Trig does.

The honest reproduction verdict on this fifth claim is therefore **partial**: our analytical model is consistent with itself and with the three filter-vs-filter rankings of the reference paper, but it does not reproduce the absolute ranking of the trigonometric method. A small empirical study with a real Arduino Nano and a wired `micros()` timing harness would settle this discrepancy in either direction.

8 Threats to Validity

Synthetic noise model. The accelerometer and gyroscope noise models are Gaussian and stationary. Real MPU-6050 silicon shows non-stationary noise (slow temperature drift, $1/f$ flicker on the gyroscope, vibration sensitivity on the accelerometer) that the synthetic pipeline does not reproduce. The reference paper's static-tilt scenario is the regime where this matters least, but a fully realistic comparison still requires hardware.

Bias floor on $|\mu_\theta|$. Because we use a single constant bias realisation across all four segments, the $|\mu_\theta|$ column of Table 3 is essentially identical for the four algorithms (they all see the same biased gravity vector and integrate it through different but similarly-tuned gains). The discriminating column is σ_θ, not $|\mu_\theta|$.

Analytical timing model. As described in Section 4.4, absolute compute-time numbers are accurate to roughly ±20%. The conclusions about the *ordering* between the three filters are robust to that band, but the Trig vs. filter ordering is not, as Section 7 discusses.

Tuning was not swept. We adopted the published parameter defaults and the published Mahony gains. A formal tuning sweep on $\mathbf{Q}$, R, β and k_p would tighten the σ_θ column further; we do not do this here so that the comparison stays close to the reference paper's setup.

Yaw is not addressed. The MPU-6050 has no magnetometer. Yaw is unobservable from gravity alone and is not reported, in agreement with the reference paper.

9 Conclusion

We have presented a simulation-based reproduction of the four-way sensor-fusion comparison of Sansone, Bartolini, Perin and Badia at ISITIA 2023 [1]. The four algorithms

(trigonometric, Kalman 1D Euler, Madgwick, Mahony) were independently re-implemented in Python, run on a synthetic MPU-6050 data stream that mirrors the reference paper's static ±15° tilt protocol, and timed on the ATmega328P with an analytical floating-point cycle-cost model. Of the five qualitative rankings that the reference paper invites, four are reproduced on our pipeline (Trig has the worst noise floor; Kalman and Madgwick are within 4% in compute; Mahony is fastest among the three filters; Kalman ties for the lowest noise floor in the accuracy table), and one diverges (the trigonometric estimator is the cheapest, not the most expensive, in our cycle-counting model). The cycle-counting divergence is a natural target for a follow-up study with a real Arduino Nano in hand.

The synthetic dataset, the four Python algorithm implementations, the analytical timing model, the experiment runner that produces Tables 3 and 4 and Figures 2–4, and the LaTeX source of this paper are released together [9] so that any third group can repeat both Badia *et al.* and our reproduction.

References

[1] S. Sansone, N. Bartolini, G. Perin, and L. Badia, "A comparative analysis of sensor fusion algorithms for miniature IMU measurements," in *Proc. International Seminar on Intelligent Technology and Its Applications (ISITIA)*, IEEE, 2023.

[2] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Trans. ASME – J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, 1960.

[3] S. O. H. Madgwick, "An efficient orientation filter for inertial and inertial/magnetic sensor arrays," Tech. Rep., University of Bristol, 2010.

[4] R. Mahony, T. Hamel, and J.-M. Pflimlin, "Nonlinear complementary filters on the special orthogonal group," *IEEE Trans. Autom. Control*, vol. 53, no. 5, pp. 1203–1218, 2008.

[5] A. M. Sabatini, "Kalman-filter-based orientation determination using inertial/magnetic sensors: Observability analysis and performance evaluation," *Sensors*, vol. 11, no. 10, pp. 9182–9206, 2011.

[6] F. L. Markley, "Multiplicative vs. additive filtering for spacecraft attitude determination," *J. Guid. Control Dyn.*, 2003.

[7] InvenSense, *MPU-6000 and MPU-6050 Product Specification, Revision 3.4*, datasheet, 2013.

[8] Atmel, *ATmega328P Datasheet*, 2015.

[9] Reproduction codebase, Marmara University EEE, `https://github.com/<placeholder>/badia-isitia2023-reproduction`, 2026.